Technical Report

PROG7312

ST10447406

Introduction

Smart-X is a.NET 8 solution that consumes simulated ESP32 sensor information, allows operators to command and reverse device operations in real time, and now represents the entire IoT mesh as a graph with quick historical searches and power-efficient routing. The three menu options (Network Topology, Command Stream, and Telemetry) are all active and connected.

Three things were included in Part 3. A time-range query only affects a small number of nodes because each sensor's timestamped history is stored in an AVL tree. Sensors, zone gateways, and the central server are modelled by a weighted graph using BFS and DFS for connectivity and failover. Prim's technique determines the minimal spanning tree that transmits telemetry to the server at the lowest overall power consumption using a binary min-heap (Cormen et al., 2022).

Because there is no actual hardware, the wireless links and their prices are created deterministically from the registered sensors rather being measured, which limits how the results can be interpreted. Since the algorithms are independent of the origin of the edges, they may eventually be replaced by actual link-quality data.

Features Implemented

Each component built upon the one before it: Part 3 reads both to construct the mesh and its history, while Part 2 repurposes the register and telemetry storage from Part 1.

In Part 1 there was the telemetry and sensor ingestion. Sensor registration with recursive deployment validation, multipart media upload, live simulated telemetry with anomaly status, Meter3 = Meter1 + Meter2 aggregation, and Docker support were all provided in Part 1. Recursion, jagged-array ring buffers, operator overloading, generics, and a multi-dimensional load matrix were all utilized.

Real-time command stream was implemented in Part 2. Standard and critical command queues, undo, a SignalR live feed, device search and filter, a per-device timeline, and recommended actions based on alarm patterns were all added in Part 2.

Queues, priority queues, stacks, dictionaries, sorted dictionaries, and hash sets were among its data structures.

In Part 3, mesh routing and network topology was introduced. A live mesh map, a route tracker by MAC or Node ID, BFS/DFS failover, a minimal spanning tree, a historical telemetry log, and a demonstration network loader were all included in Part 3. Prim's algorithm, BFS, DFS, a graph, an AVL tree, and a binary min-heap were among its data structures.

The Network Topology page specifically addresses each need for Part 3:

- The active mesh dashboard is an SVG map that shows the server, zone gateways, sensors, and failing nodes.
- Tracking by MAC or identifier: the route tracker determines the hop-by-hop path to the server based on a MAC address, Node ID, or sensor ID.
- Historical log: a range query on a chosen sensor that yields a sparkline, summary data, and a table.
- Tree requirement: "all readings between 14:00 and 16:00 for Node A" is a range query rather than a list scan since each sensor's readings are stored in an AVL tree keyed on timestamp.
- Graph, traversal, and MST requirements: Prim's algorithm provides the power-efficient path to the server; BFS and DFS locate connected devices and the hops around down nodes; the mesh is a weighted graph.

Architecture

There are three projects in the solution. Every few seconds, the Blazor client polls the API and receives real-time command events using SignalR (Microsoft, 2026d). SmartX is called via the API's services (the registry and sensor simulator are not shown).shared, where Part 3 introduces the AVL tree, min-heap, and mesh graph. A response shape is defined in one location since both the client and the API make reference to the shared DTOs.

Task 3 Challenges and The Outcome

The mesh without radios was challenging as there is no actual link data to create a graph from because the sensors are simulated. I created the topology deterministically:

every zone has a gateway connected to the server, every sensor connects to both its gateway and its closest neighbors, and neighboring zones share a bridge link. Instead of using a string, link charges are derived from a FNV-1a hash (Fowler et al., 2025) of the MAC addresses.Without GetHashCode, which.NET randomly assigns to each process, the graph and, thus, the MST would vary with each restart.

Failure without reconstructing the graph was also extremely difficult. If an operator designates a node as down or if its telemetry has been silent for fifteen seconds (the current Disconnected status from Part 1), the node is considered down. BFS, DFS, and Prim's simply bypass dead vertices because each algorithm accepts a passing condition rather than changing the graph. The page can display "rerouted around ESP32-A2" and the additional power consumption since the route tracker calculates the route for both a healthy mesh and the present one.

The AVL tree has identical timestamps. While a batch ingest can stamp several readings with the same instant, a binary search tree requires unique keys. An insertion sequence number and UTC ticks are combined to create the key. Long is then used in range queries.Long and MinValue.Every reading on the boundary tick is included because MaxValue is the sequence bounds. Inserts from the simulator and queries from the page are kept consistent via a per-sensor lock.

Connecting the Blazor frontend to the C# backend. There were three issues that were limited to the browser. Every number passes via CultureInfo because SVG attributes produced from a double render with a comma on a South African (en-ZA) locale quietly damage the map (Microsoft, 2026c). SVG labels conflict with Razor's treatment of as its own pseudo-tag, thus such elements are contained within a <g>. The page now just re-traces the most recent identifier that was submitted because the three-second refresh was re-tracing whatever was partially written in the search field.

Learnings I Have Discovered

A project can be reviewed with small, named commits. It was simple to determine which layer each change belonged to and to safely order dependent changes by using conventional-commit messages with one change per commit (for instance, feat(shared): add per-sensor history index backed by AVL trees).

From the access pattern, select the structure. An AVL tree responds to range queries over time in O(log n + k) for Task 3 (Cormen, et al., 2022). The saving over a complete scan is evident, and it increases as the history does. The display shows how many tree nodes each query hit next to how many readings are stored. The identical concept was covered in Part 2 using a dictionary for O(1) MAC lookup and a SortedDictionary for the device timeline.

Comparing the results with an impartial oracle. It is difficult to eyeball a minimum spanning tree. Compared to a single hand-written example, writing Kruskal's algorithm separately and verifying that its total cost equals Prim's on 40 random meshes provided significantly greater confidence (Cormen et al., 2022). After sorted and shuffled inserts, the AVL tree's IsValidAvl method verifies ordering, stored heights, and balancing factors.

Technology Recommendations

TECHNOLOGY	MOTIVATION
POSTGRESQL WITH TIMESCALEDB, HOSTED ON GOOGLE CLOUD SQL	Managed PostgreSQL on Cloud SQL provides robust storage for sensors, telemetry, and command history (Google Cloud, 2026). A time-series database (Timescale, 2026) maintains history across restarts and many API instances, and its time-bucketed queries replicate what the AVL range query does in memory. Currently, SensorRegistry and the AVL history are in memory and reset at restart. IAM-controlled access, private IP networking, automated backups, and encryption both in transit and at rest are all added for security. Time partitioning and read replicas allow it to grow, while indexed time-bucketed queries prevent full scans.
GITHUB ACTIONS CI/CD	With GitHub Actions (GitHub, 2026), the two Docker images are built and may be deployed upon merge with each push that executes dotnet build and dotnet run --project tests/SmartX.Tests. Since the test project is a simple console application, a non-zero exit code causes

	the pipeline to fail without the need for additional tools. Security checks are automatic since scanning may be done on each push and encrypted secrets remain out of the source. Because jobs operate in parallel, regressions are detected in a matter of minutes and a single pipeline may scale to every service.
LLM API FOR DAILY ALERT SUMMARIES	The alarms and command chronology for the day are sent to a model by a scheduled job, which also records a brief summary in plain English (e.g., "Zone B power draw spiked twice; ESP32-B2 went silent for 40 minutes"). Rather than monitoring the live video, operators read a single text. Little data leaves the system because only aggregated alert text is delivered and the API key is kept in a hidden store. It operates as a background job off the request path and makes one call every day, regardless of the number of devices.
HOME ASSISTANT INTEGRATION	Allows Home Assistant automations to respond to orders by exposing Smart-X sensors and actuators in a home or building automation dashboard (Home Assistant, s.a.). The automation load resides in Home Assistant rather than the API, a scoped, read-only token restricts what automations can do, and MQTT discovery pushes changes rather than polling.
REDIS BACKPLANE FOR SIGNALR	There is presently only one server where the Command Stream's live stream operates. Horizontal scalability begins with a Redis backplane that allows several API instances to share broadcasts. Clients receive events from any instance they contact thanks to Redis's low-latency publish/subscribe system, which operates on a password-

	protected private network using TLS (Microsoft, 2026e).
OPENTELEMETRY WITH PROMETHEUS AND GRAFANA	Performance claims may be measured rather than inferred thanks to metrics and traces for intake rate, queue depth, command latency, and topology query time (OpenTelemetry, 2026) . Vendor-neutral instrumentation scalable from one service to a fleet, metrics don't carry any sensitive payloads, and measurements reveal where time is spent.
AUTHENTICATION WITH ASP.NET CORE IDENTITY OR MICROSOFT ENTRA ID (JWT)	These days, commands can be issued by anyone who can access the API. The command and node-failure endpoints would be safeguarded by role-based access (viewer, operator, administrator). One central identity serves all instances, roles and expiration tokens strengthen security, and stateless token validation eliminates the requirement for session lookup (Microsoft, 2026a).
CONTAINERS ON GOOGLE CLOUD RUN	Uses controlled container orchestration in place of a manually executed docker-compose file to run the current API and Blazor Docker images as managed containers with autoscaling and rolling revisions (Google Cloud, 2026b). Rolling revisions deploy without any downtime, autoscaling (even to zero) manages fluctuating demand, and managed patching, per-service IDs, and IAM enhance security.
CLOUD INGESTION PIPELINE: MQTT OR HTTP INTO PUB/SUB, WITH BIGQUERY FOR ANALYSIS	A Pub/Sub topic is where devices publish (Google Cloud, 2026d). While a second subscription streams readings into BigQuery (Google Cloud, 2026a) for trend and anomaly queries across all zones, a hosted service subscribes and calls the current ingest path. BigQuery expands

	without servers, Pub/Sub absorbs bursts, IAM per topic and default encryption protect the data, and input and processing are separated so analytics never slow down the live store.

The entire process from a sensor reading to an operator decision is now covered by Smart-X, including ingesting, commanding, and comprehending the network that transmits the data. Each of the Part 3 data structures: an AVL tree for history, a graph for the mesh, and a heap-backed Prim's algorithm for the cheapest routes performs a particular function in the Network Topology feature. The primary remaining step is to replace simulated devices and in-memory storage with actual hardware and a database, as previously suggested.

Reference List

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms*. 4th ed. Cambridge, MA: MIT Press.

GitHub, 2026. *GitHub Actions documentation*. [online] Available at: <https://docs.github.com/en/actions> [Accessed 01 November 2026].

Google Cloud, 2026. *Cloud SQL for PostgreSQL documentation*. [online] Available at: <https://cloud.google.com/sql/docs/postgres> [Accessed 10 November 2026].

Home Assistant, s.a. *Home Assistant*. [online] Available at: <https://www.home-assistant.io/> [Accessed 10 November 2026].

Microsoft, 2026a. *Introduction to authorization in ASP.NET Core*. [online] Available at: <https://learn.microsoft.com/aspnet/core/security/authorization/introduction> [Accessed 11 November 2026].

Microsoft, 2026b. *Background tasks with hosted services in ASP.NET Core*. [online] Available at: <https://learn.microsoft.com/aspnet/core/fundamentals/host/hosted-services> [Accessed 11 November 2026].

Microsoft, 2026c. *CultureInfo class*. [online] Available at: <https://learn.microsoft.com/dotnet/api/system.globalization.cultureinfo> [Accessed 11 November 2026].

Microsoft, 2026d. *Overview of ASP.NET Core SignalR*. [online] Available at: <https://learn.microsoft.com/aspnet/core/signalr/introduction> [Accessed 11 November 2026].

Microsoft, 2026e. *Set up a Redis backplane for ASP.NET Core SignalR scale-out*. [online] Available at: <https://learn.microsoft.com/aspnet/core/signalr/redis-backplane> [Accessed 11 November 2026].

OpenTelemetry, 2026. *Getting started: .NET*. [online] Available at: <https://opentelemetry.io/docs/languages/dotnet/getting-started/> [Accessed 11 November 2026].

Timescale, 2026. *Hypertables*. [online] Available at: <https://docs.timescale.com/use-timescaledb/latest/hypertables/> [Accessed 11 November 2026].